

Information Retrieval 5

Transformer and BERT for Ranking

Makoto P. Kato, National Institute of Informatics / University of Tsukuba

This lecture is based on the following textbook:

Jimmy Lin, Rodrigo Nogueira, Andrew Yates. *Pretrained Transformers for Text Ranking: BERT and Beyond*. Synthesis Lectures on Human Language Technologies, Springer, 2021.

Boolean Model: Exact matching based on set theory. No ranking capability

Vector Space Model: TF-IDF weighting + cosine similarity. Weak theoretical foundation

Probabilistic Model: BIM based on PRP leading to BM25. Still the standard baseline today

Language Model: Probability of a document generating a query. Smoothing implicitly achieves IDF-like effects

In this lecture, we will learn the **Transformer / BERT** architecture and training techniques for ranking tasks that go beyond the limitations of traditional models

Gain a deep understanding of Transformer and BERT as the foundation of neural retrieval models, and learn training techniques for applying them to ranking tasks

First Half: From Neural Networks to Transformer and BERT

- Fundamentals of neural networks
- Word embeddings (distributed representations)
- Attention and Self-Attention
- Transformer architecture
- BERT pre-training and input representation

Second Half: Training for Ranking

- Basics of fine-tuning
- Loss functions for ranking
- Negative sampling strategies
- Additional pre-training and multi-step training
- MS MARCO training pipeline

Fundamentals of Neural Networks

Deep Learning for NLP

An **artificial neuron (perceptron)** generates an output by applying an activation function to the weighted sum of inputs

$$y = f \left(\sum_{i=1}^n w_i x_i + b \right)$$

x_i : input, w_i : weight, b : bias, f : activation function

Weights w_i are **learned** from data
(backpropagation)

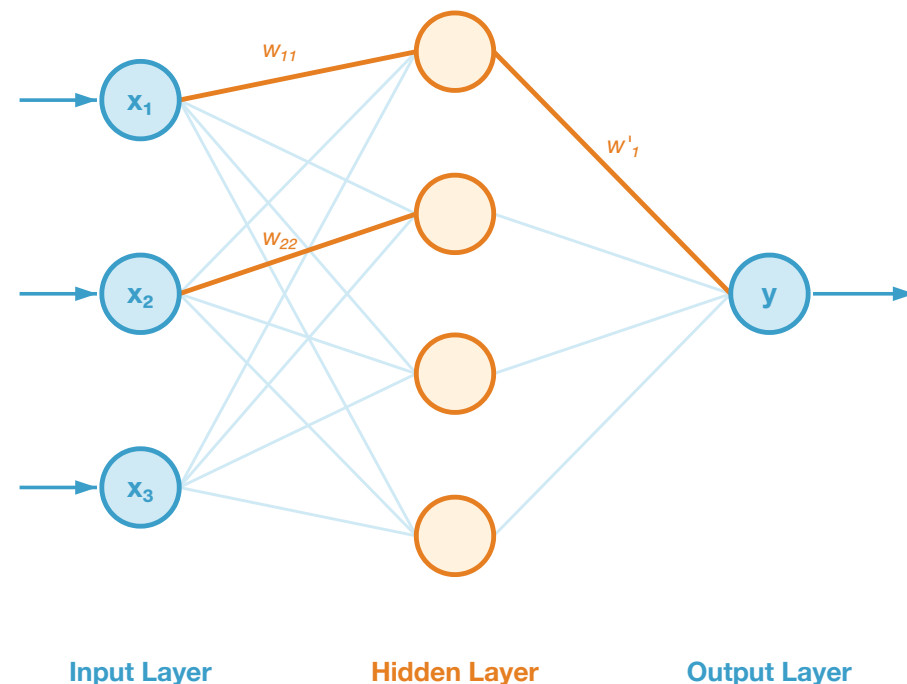
A **multi-layer perceptron (MLP)** stacks multiple layers:

Input layer → **Hidden layer(s)** (one or more) →

Output layer

Deeper layers can learn more complex patterns →
deep learning

Multi-Layer Perceptron (MLP)



Each connection between nodes has a weight w that is learned from training data. The number of hidden nodes and layers are hyperparameters

Without activation functions, even multi-layer networks are merely **compositions of linear transformations = a linear transformation**

$$f(Wx + b) \xrightarrow{f=\text{identity}} Wx + b$$

Non-linear activation functions enable approximation of increasingly complex functions as layers are stacked

Key activation functions:

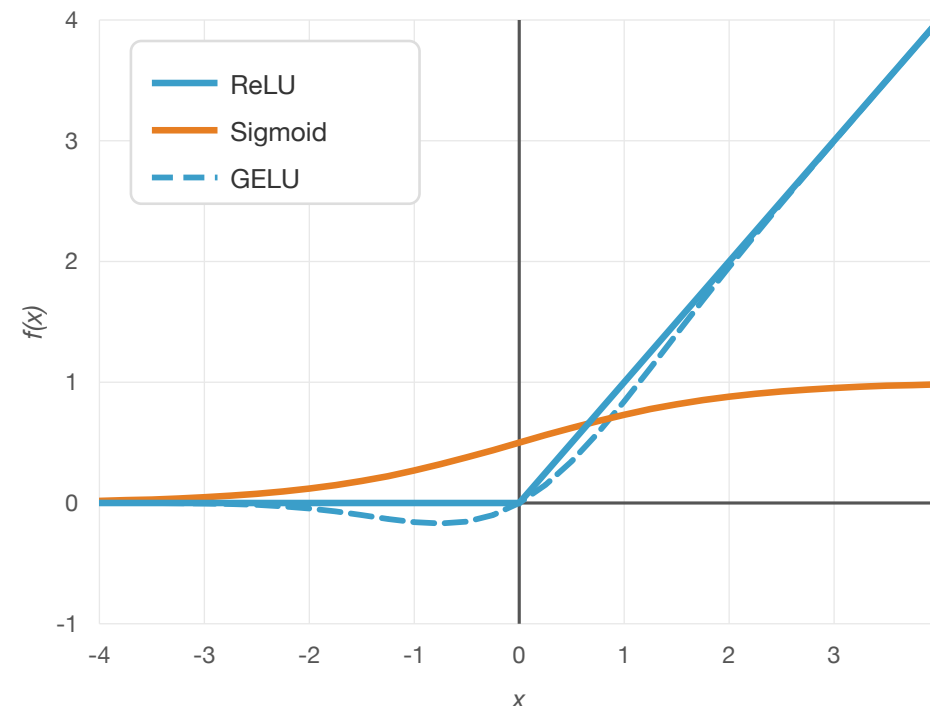
ReLU: $f(x) = \max(0, x)$ — Fast computation, less gradient vanishing. Standard in CNNs and MLPs

Sigmoid: $f(x) = \frac{1}{1+e^{-x}}$ — Squashes output to $[0, 1]$. Used in output layers for binary classification

GELU: $f(x) = x \cdot \Phi(x)$ — Adopted in BERT and GPT. A smooth approximation of ReLU

$\Phi(x)$: CDF of the standard normal distribution (x large $\rightarrow 1$, small $\rightarrow 0$)

Activation Functions



The Feed-Forward layers in Transformer / BERT use **GELU**. It avoids the "discontinuity at 0" problem of ReLU while having a probabilistic interpretation. The GELU graph is shown overlaid with ReLU (orange dashed line) for comparison

In NLP, words are represented as **fixed-dimensional real-valued vectors**. This is called an **embedding**

Problems with one-hot representation:

100K vocabulary $\rightarrow$ 100K-dimensional sparse vector

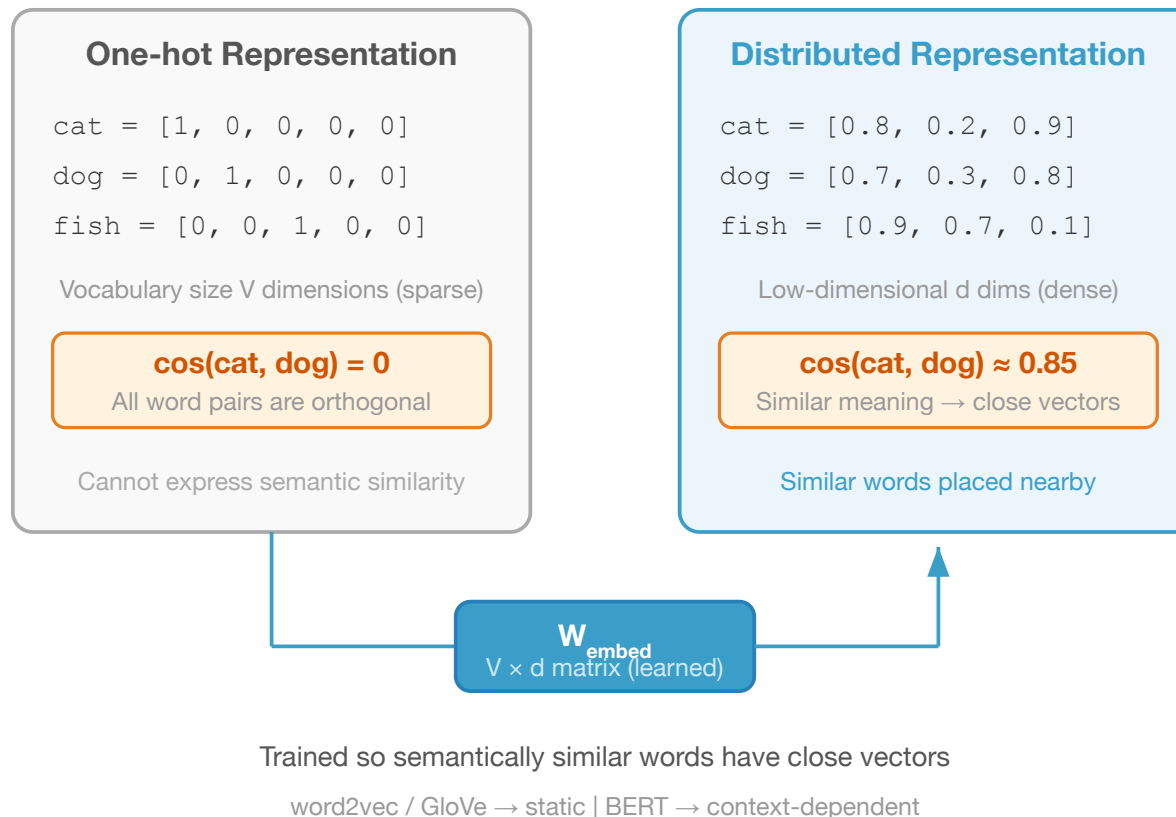
Cannot represent relationships between words: $\cos(\text{"Kyoto"}, \text{"Nara"}) = 0$

Advantages of dense embeddings:

Low-dimensional (100–768 dimensions) dense vectors

Learned so that **semantically similar words have similar vectors**

$\cos(\text{"Kyoto"}, \text{"Nara"}) \approx 0.85$ (high similarity)



word2vec [Mikolov et al., 2013]:

Learns embeddings by predicting a target word from surrounding words (Skip-gram) or vice versa (CBOW)

First large-scale realization of "semantic similarity $\approx$ vector proximity"

Limitation: One vector per word $\rightarrow$ **Cannot distinguish polysemous words**

Contextualized Embeddings:

Generate **different vectors for the same word depending on context**

ELMo [Peters et al., 2018] $\rightarrow$ **BERT** [Devlin et al., 2019]

Important for IR: Can distinguish "bank (financial)" from "bank (river)" by context

Static vs Contextualized Embeddings

word2vec (Static)

"bank in **account**" $\rightarrow \mathbf{v}_{\text{account}}$

"river **account**" $\rightarrow \mathbf{v}_{\text{account}}$ (same!)

BERT (Contextualized)

"**bank** account" $\rightarrow \mathbf{h}_{\text{account}}^{(1)}$ financial context

"**cooking** account" $\rightarrow \mathbf{h}_{\text{account}}^{(2)}$ different context

$\rightarrow$ The two vectors are **different**

Each layer of BERT progressively integrates contextual information, producing sophisticated semantic representations in the final layer

Attention originated in machine translation

[Bahdanau et al., 2015]

Problem: Compressing a long sentence into a fixed-length vector with an RNN causes information from the beginning to be lost

Solution: At each step, the decoder **refers to all positions in the encoder** and **"attends to" positions with high relevance**

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}$$

α_{ij} : attention weight that decoder position i places on encoder position j

e_{ij} : relevance score between positions i and j

Key insight: The model can **automatically** learn "which parts of the sequence are important" from data

Example of Attention in Translation

Input (English): "The cat sat on the mat"

Output (Japanese): "猫がマットの上に座った"

When generating "猫" (cat):

The **cat** sat on the mat

→ A high attention weight is assigned to "cat"

Self-Attention in the Transformer is an extension of this Attention idea, applied within the same sequence

Transformer

Limitations of previous models:

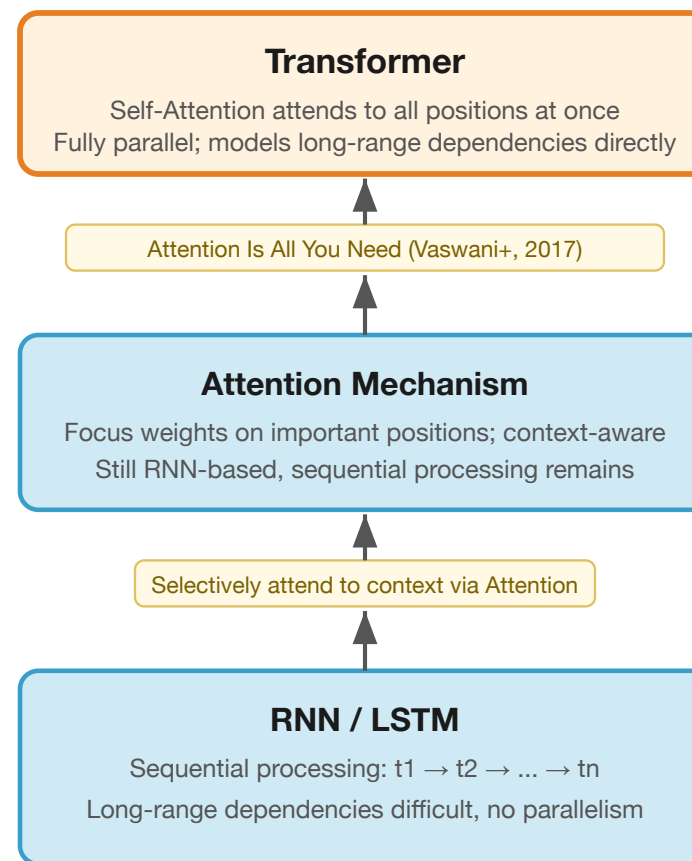
DSSM: Independent encoding → no term interactions

DRMM: Similarity matrix but no context

RNN/LSTM: Sequential, gradient vanishing, hard to parallelize

Transformer [Vaswani et al., 2017]:

Self-Attention captures full-sequence context simultaneously
Parallel computation, directly models long-range dependencies



Computational complexity comparison:

RNN: $O(n)$ steps for sequence length n (sequential)

Transformer: $O(1)$ steps (parallel), but each step $O(n^2)$

Self-Attention is a mechanism where **each token in the input sequence computes its relevance to all other tokens**, generating context-dependent representations

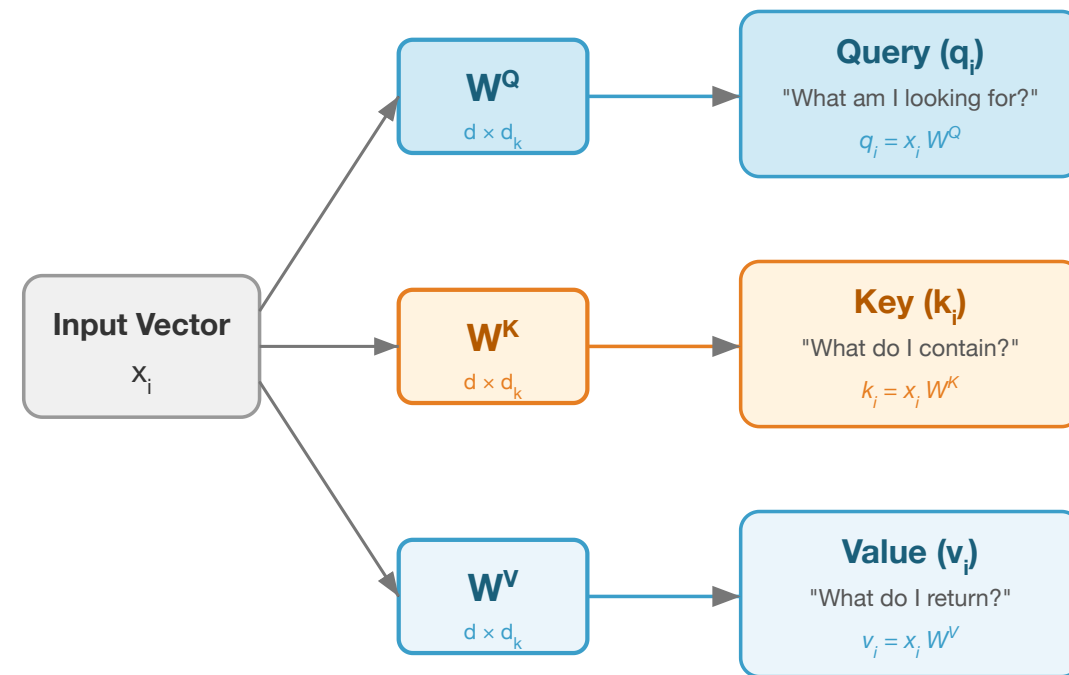
From each token's input vector x_i , three vectors are generated:

Q (Query): "What am I looking for?" — $q_i = x_i W^Q$

K (Key): "What do I have?" — $k_i = x_i W^K$

V (Value): "What do I return?" — $v_i = x_i W^V$

Intuition: Each token "asks a question (Q)" to other tokens, and aggregates "values (V)" according to the match with "keys (K)"



Each token independently produces Q, K, V (weight matrices shared across all tokens)

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Input: "Kyoto no koyo" (3 tokens, simplified example with $d_k = 2$)

Step 1: Generate Q, K, V from each token

	Q	K	V
Kyoto	[1, 0]	[0, 1]	[1, 1]
no	[0, 1]	[1, 0]	[0, 1]
koyo	[1, 1]	[1, 1]	[1, 0]

Step 2: Dot product of Q and K → score matrix

$$QK^T = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 2 \end{pmatrix}$$

Step 3: Scale by $\sqrt{d_k} = \sqrt{2} \approx 1.41$ and apply softmax

$$\frac{QK^T}{\sqrt{d_k}} \approx \begin{pmatrix} 0 & 0.71 & 0.71 \\ 0.71 & 0 & 0.71 \\ 0.71 & 0.71 & 1.41 \end{pmatrix}$$

Softmax of the "koyo" row: [0.25, 0.25, 0.50]

→ "koyo" **attends most to itself** while also referencing "Kyoto" and "no"

Step 4: Weighted sum of V using attention weights

$$\begin{aligned} \text{out}_{\text{koyo}} &= 0.25 \cdot [1, 1] + 0.25 \cdot [0, 1] + 0.50 \cdot [1, 0] \\ &= [0.75, 0.50] \end{aligned}$$

In actual BERT, $d_k = 64$, sequence length = 512, but the principle is the same

Why divide by $\sqrt{d_k}$?

The dot product of Q and K grows in scale as the dimensionality d_k increases

Assuming each element is an independent random variable with mean 0 and variance 1:

The variance of $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ is proportional to d_k

For $d_k = 64$, the standard deviation of the dot product ≈ 8

Problem: When dot product values are large, softmax produces an **extreme distribution** (concentrated on one, others nearly 0)

$$\text{softmax}([10, 1, 1]) \approx [0.9999, 0.0001, 0.0001]$$

→ Gradients become nearly 0, preventing learning

Solution: Dividing by $\sqrt{d_k}$ normalizes the variance of the dot product to 1

Effect of Scaling

Without scaling ($d_k = 64$)

Dot products: [52.3, 3.1, -8.7]

softmax: [**1.000**, 0.000, 0.000]

→ Attends to only one token (hard attention)

→ Gradient vanishing makes learning difficult

With scaling ($\div \sqrt{64} = 8$)

After scaling: [6.5, 0.4, -1.1]

softmax: [**0.997**, 0.002, 0.001]

→ Smoother distribution

→ Gradients flow, learning stabilizes

A single Attention can only capture one type of relationship → **Run multiple Attentions in parallel**

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

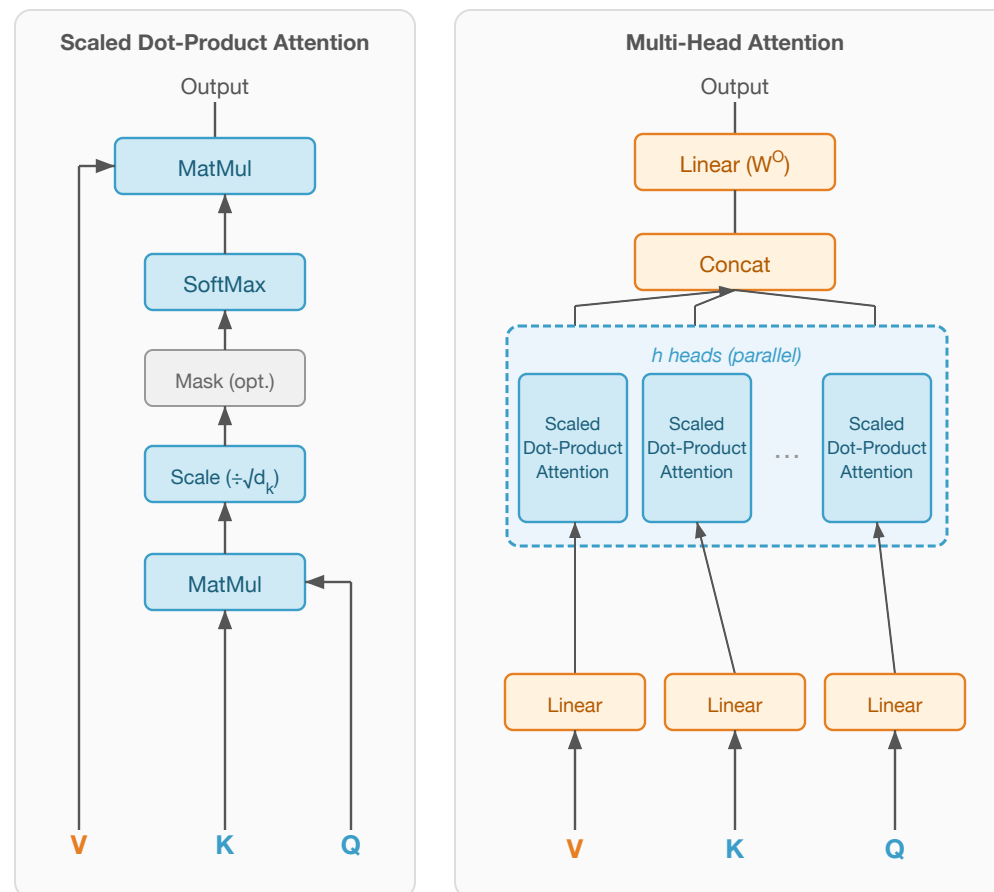
h : Number of heads (BERT-Base:
 $h = 12$)

Each head learns a **different type of relationship**

One head: Syntactic relationships
(subject-predicate)

One head: Semantic relationships
(synonyms)

One head: Positional proximity



Each head operates on $d_k = d_{\text{model}} / h = 768 / 12 = 64$ dimensions, so the computational cost is roughly the same as a single full attention head

Self-Attention **does not distinguish input order** (set function) → need **Positional Encoding**

Original Transformer — Sinusoidal:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

pos: position, *i*: dimension index. Different wavelengths per dimension

BERT — Learnable Position Embedding: learnable parameter per position (max 512)

Intuition for Positional Encoding

Input embeddings (without position info):

cat sat down

+ Positional encoding:

pos=0 pos=1 pos=2

= Position-aware embeddings:

cat@0 sat@1 down@2

Sinusoidal vs learnable achieve similar performance, but BERT cannot exceed 512 tokens

Two techniques to prevent **gradient vanishing** in deep networks:

Residual Connection: $\text{output} = x + \text{SubLayer}(x)$

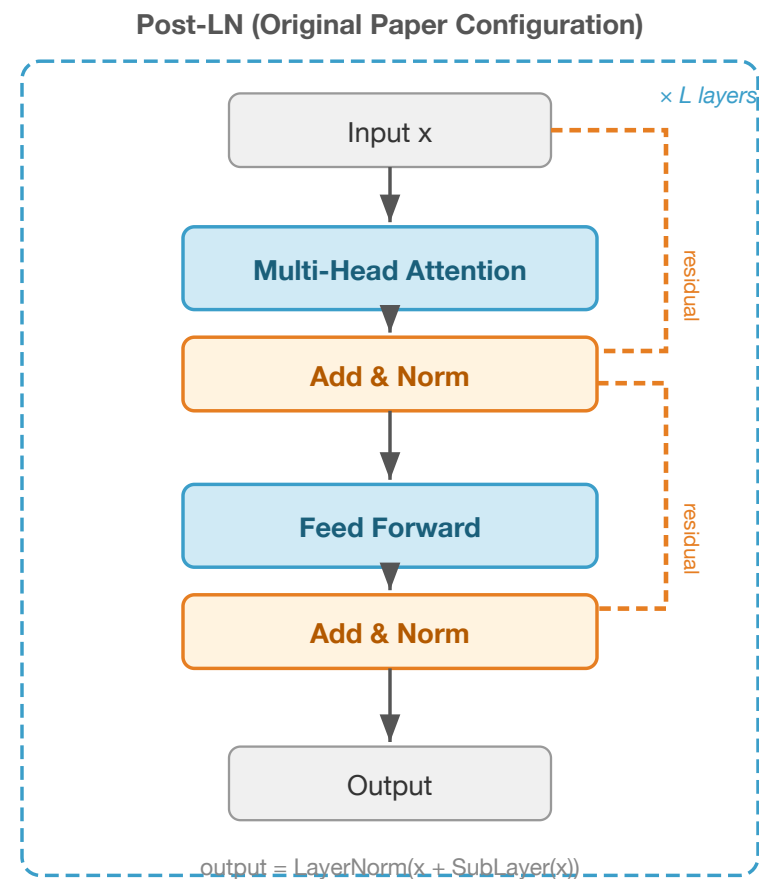
— Adds input directly (shortcut), enabling stable deep learning

Layer Normalization:

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sigma} \cdot \gamma + \beta$$

— Normalizes per token (mean 0, var 1). γ, β learnable. Stabilizes training

BERT: **Post-LN** (Add $\rightarrow$ LN). GPT-2+: **Pre-LN** (LN $\rightarrow$ SubLayer $\rightarrow$ Add)



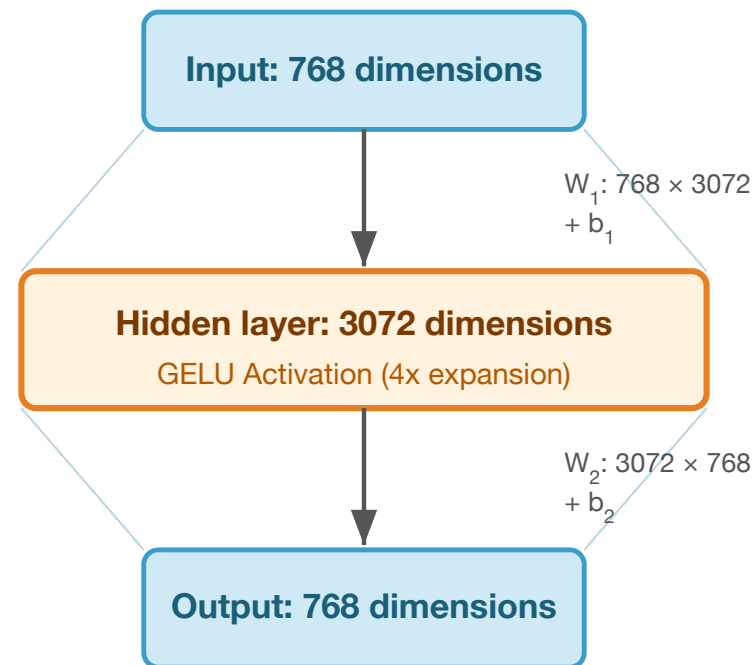
He et al., "Deep Residual Learning for Image Recognition", CVPR, 2016; Ba et al., "Layer Normalization", 2016

A 2-layer fully connected network applied **independently to each token**

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2$$

Input $\rightarrow$ intermediate ($d_{ff} = 3072$, $4\times$ expansion) $\rightarrow$ output (768 dims)

Role: Self-Attention determines "what to gather"; FFN determines "**how to transform**" the gathered information. FFN also functions as a **knowledge memory store**



$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2$$

Applied independently to each token position

Parameters per layer in BERT-Base:
Self-Attention $\approx$ **2.4M** / FFN $\approx$ **4.7M** (about 2/3 of the total)

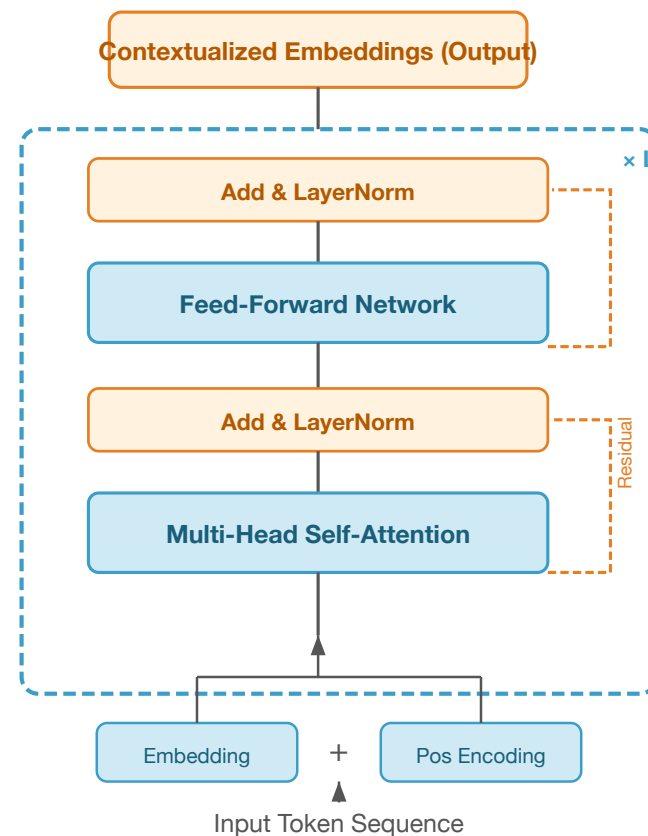
One Encoder layer = 2 sub-layers + Residual + LayerNorm:

1. **Multi-Head Self-Attention:** $z = \text{LN}(x + \text{MultiHead}(x))$

2. **Feed-Forward Network:** $\text{out} = \text{LN}(z + \text{FFN}(z))$

Stacked for **L layers** (lower → syntax, upper → semantics):

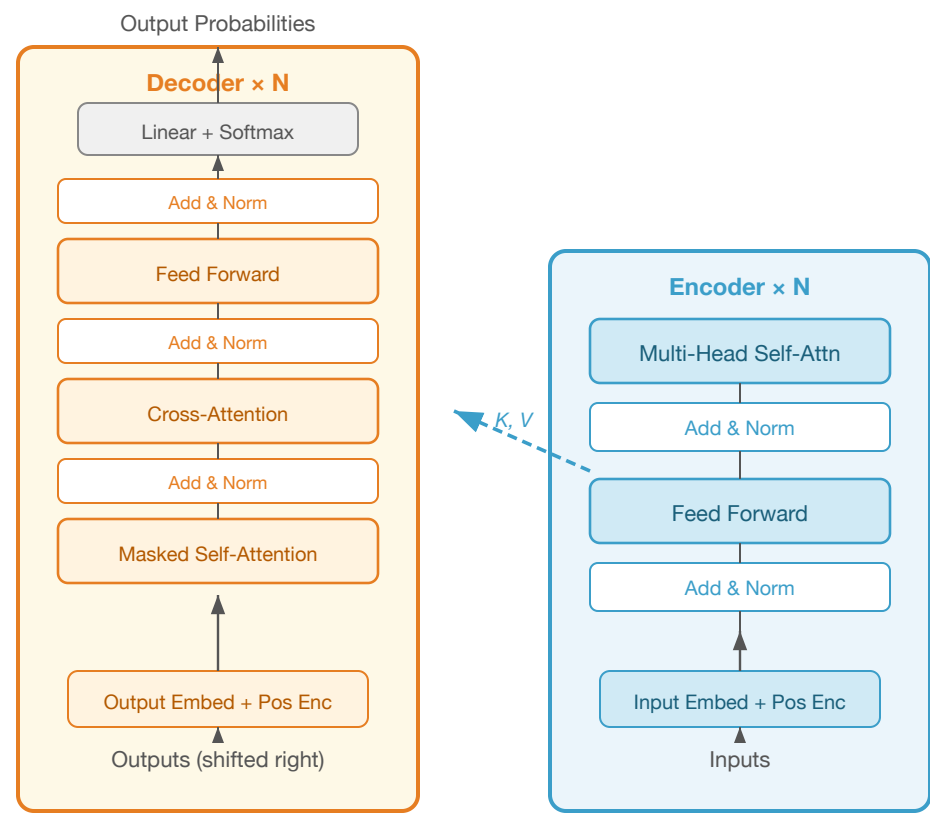
Model	L	Hidden	Heads	Params
Base	12	768	12	110M
Large	24	1024	16	340M



BERT-Base: L=12, d=768 | BERT-Large: L=24, d=1024

BERT-Base: L = 12, BERT-Large: L = 24

Vaswani et al., "Attention Is All You Need", NeurIPS, 2017



Vaswani et al., "Attention Is All You Need", NeurIPS, 2017

The original Transformer has an **Encoder-Decoder** structure

Component	Feature	Derivative
Encoder (blue)	Bidirectional Self-Attn	BERT
Decoder (orange)	Masked + Cross-Attn	GPT
Both	Enc-Dec	T5

Each sub-layer has **Add & LayerNorm** (residual connection)

Usage in retrieval:

- BERT** → Cross-encoder reranking
- GPT** → Listwise reranking
- T5** → monoT5 / duoT5

Vaswani et al., "Attention Is All You Need", NeurIPS, 2017

Comparison with pre-Transformer models:

Property	RNN/LSTM	CNN	Transformer
Capturing long-range dependencies	Difficult (gradient vanishing)	Limited by kernel size	Direct reference ($O(1)$ path)
Parallel computation	Impossible (sequential)	Possible	Fully parallel
Bidirectional context	Possible with BiLSTM but slow	Possible	Naturally achieved via Self-Attention
Computational complexity	$O(n \cdot d^2)$	$O(k \cdot n \cdot d^2)$	$O(n^2 \cdot d)$

Transformer complexity $O(n^2 \cdot d)$: Expensive for long sequences. BERT max 512 tokens. Efficient variants: Longformer, BigBird

Transformer = Self-Attention + Positional Encoding + Residual Connection + LayerNorm + FFN. Each component was known, but their integration revolutionized NLP

BERT

Bidirectional Encoder Representations from Transformers

Traditional approach (~2017):

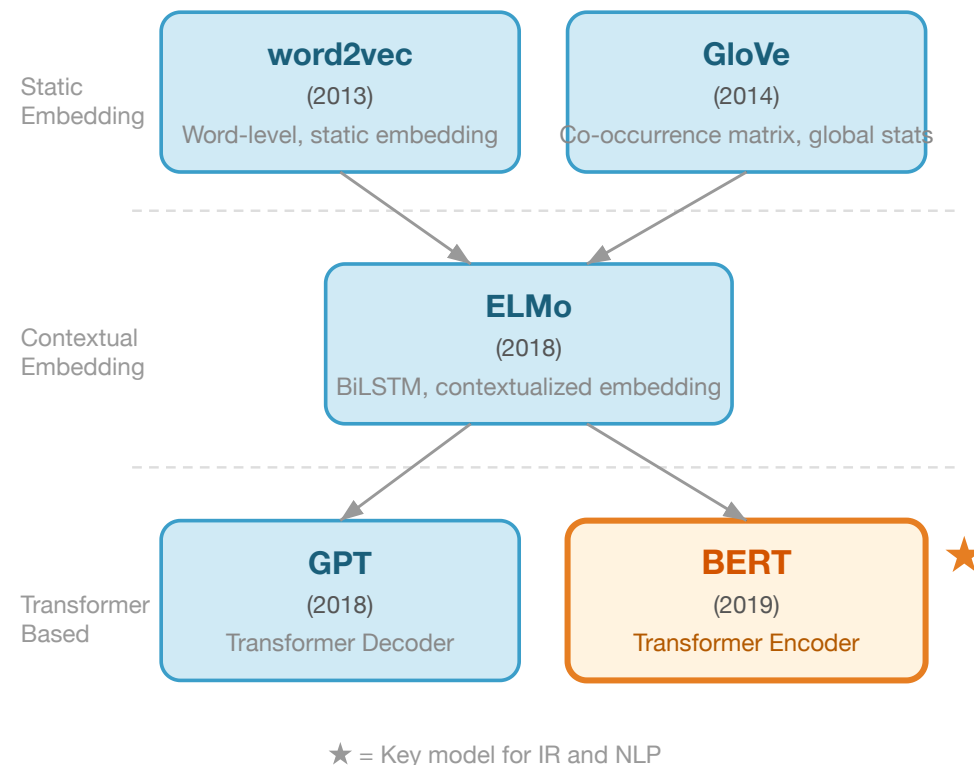
Train from scratch for each task;
requires large labeled data; no
knowledge sharing

Pre-training paradigm (2018~):

1. Learn general language knowledge from **large-scale unlabeled data** (pre-training)
2. Fine-tune with **small task-specific data** (fine-tuning)

Why it works: Language structure (grammar, semantics, world knowledge) is **task-independent** — once learned, it transfers to any task

Evolution of Pre-trained Models



BERT [Devlin et al., 2019] = A **contextualized embedding model** that leverages the Encoder portion of the Transformer

The innovation was combining the Transformer Encoder with **self-supervised learning**

Input: token sequence → Output: **context-dependent** representation for each token

Differences from word2vec / GloVe:

word2vec: "bank" always has the same vector

BERT: "river bank" and "bank account" → **different vectors**

BERT Model Sizes

Config	Layers	Hidden dim	Heads	Parameters
Base	12	768	12	110M
Large	24	1024	16	340M

Google released pre-trained models and code → rapid community adoption. Hugging Face Transformers is the de facto standard

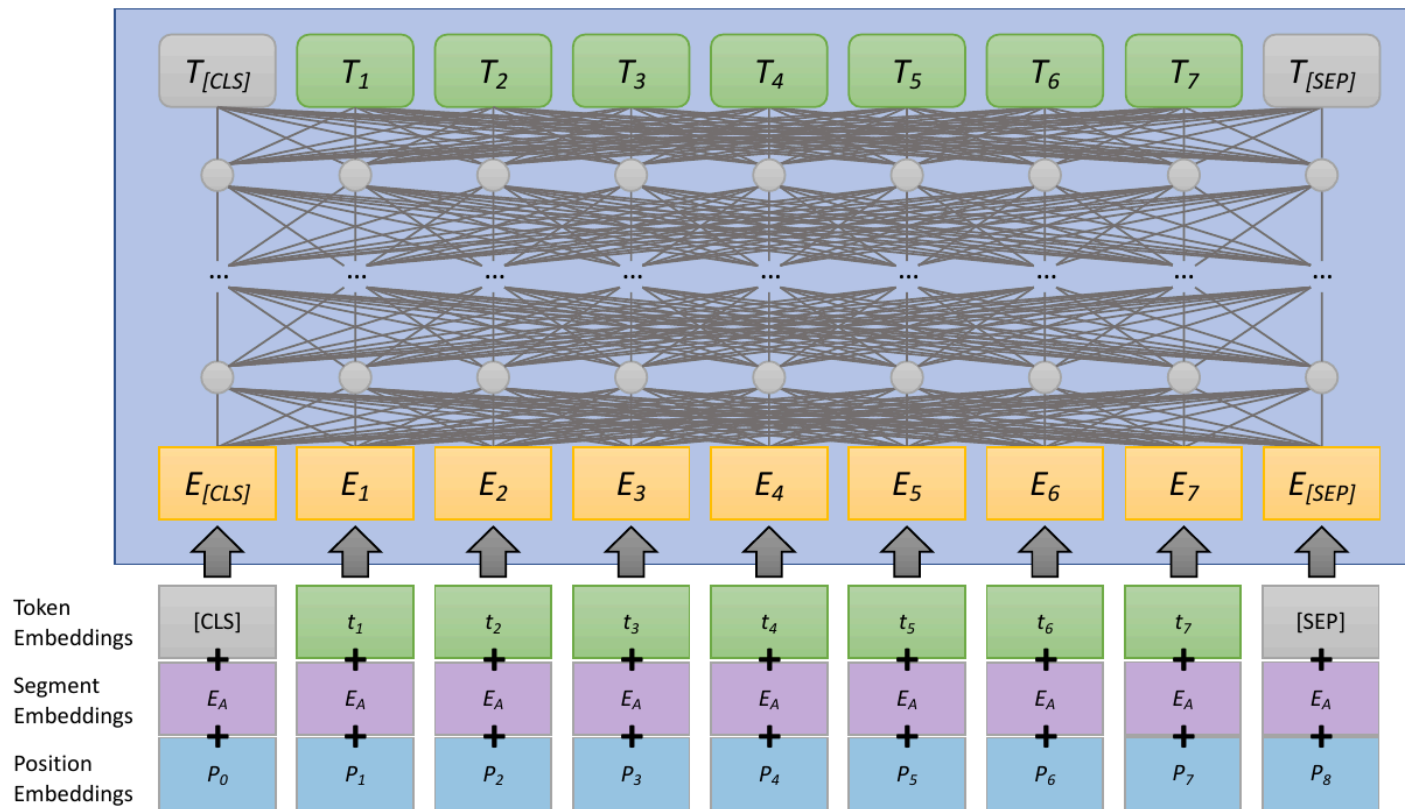
BERT uses the Transformer Encoder **as is**:

1. Convert input tokens into **embeddings**
2. **Contextualize** through L Transformer Encoder layers
3. Output **contextualized embeddings** for each token

Input: [CLS] t_1 t_2 ... t_n [SEP] →

Output: h_i per token (768 dims)

[CLS] token: Aggregate representation of the entire sequence, used for classification and ranking



Source: Lin et al. (2021), Figure 4

Each token's input representation is the **sum of three embeddings**:

1. **Token Embedding**: Subword embedding from the WordPiece tokenizer
2. **Segment Embedding**: Indicates whether the input belongs to Segment A or B
3. **Position Embedding**: Token position information (learnable)

Special tokens:

[CLS]: Aggregate representation of the entire sequence (used in classification tasks)

[SEP]: Separator between two inputs

Composition of Input Representation

Token	[CLS]	Kyo	to	[SEP]	au	tumn	[SEP]
Segment	A	A	A	A	B	B	B
Position	0	1	2	3	4	5	6

The three embeddings are combined by **element-wise addition** (not concatenation). The maximum sequence length is 512 tokens

A subword segmentation that keeps vocabulary size at approximately **30,000** while enabling representation of any text

Eliminates the unknown word problem: low-frequency words are split into subwords

The `##` prefix indicates concatenation with the preceding token

Segmentation examples:

Input word	WordPiece
scrolling	scroll + ##ing
biking	bi + ##king
information	information (no split)

Impact on IR

WordPiece has **no relation** to linguistic morphemes

"biostatistics" → "bio" + "##sta" + "##tist" + "##ics"

(does not align with morphemes)

Implications for retrieval:

BERT processes text at the **subword level**. Word-level meaning is recovered from contextualized embeddings of subwords

The 512-token limit is a limit on **subword count**, not word count

MLM (Masked Language Model):

Randomly mask **15%** of input tokens and predict the original tokens

Learns the structure of language by solving "fill-in-the-blank" problems

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \text{masked}} \log P(t_i \mid \text{context})$$

Masking strategy (for the 15% of tokens selected for masking):

80%: Replace with [MASK] token

10%: Replace with a random token

10%: Leave unchanged (as is)

MLM Example

Input:

"The [MASK] leaves in Kyoto are best in November"

Prediction:

$P(\text{autumn} \mid \text{context}) = 0.72$

$P(\text{cherry} \mid \text{context}) = 0.15$

$P(\text{flower} \mid \text{context}) = 0.08$

...

What is learned:

Relationship between word meaning and context

Context-dependent usage of polysemous words

Syntactic constraints

Correctly predicting masked words requires **deep linguistic knowledge**:

Learning syntax:

"The cat [MASK] the fish" → A verb is needed
(part-of-speech understanding)

"She [MASK] the book" → "read", "bought"
(argument structure understanding)

Learning semantics:

"The doctor [MASK] the patient" → "examined",
"treated" (conceptual relationships)

"Tokyo is the [MASK] of Japan" → "capital" (world knowledge)

Pragmatic knowledge:

Inferring appropriate words from context requires co-occurrence patterns, causal relationships, and common-sense knowledge

Hierarchy of Knowledge Learned by MLM

Lower layers (1-4)

Part-of-speech, local syntactic patterns
"[MASK] runs" → A noun is needed

Middle layers (5-8)

Dependency parsing, semantic roles
"The doctor [MASK] the patient" → "treats"

Upper layers (9-12)

Task-specific semantic representations
Sentence-level meaning, entailment relations

Each layer of BERT progressively extracts information at higher levels of abstraction (confirmed by probing studies)

NSP (Next Sentence Prediction):

A binary classification task predicting whether two sentences are consecutive

Determined from the output of the [CLS] token

Training data composition:

50%: Actually consecutive pairs (positive)

50%: Random pairs (negative)

Purpose: Acquire inter-sentence understanding (important for retrieval)

NSP Examples

IsNext (Positive)

A: "Kyoto is the ancient capital of Japan"

B: "Many temples and shrines remain there"

NotNext (Negative)

A: "Kyoto is the ancient capital of Japan"

B: "Research on quantum computers is progressing"

RoBERTa questioned NSP's necessity, but it likely helped BERT's sentence-pair understanding

Pre-training corpus (~3.3B words total):

BooksCorpus: ~800M words (11K books)

English Wikipedia: ~2.5B words

Training: Batch 256, seq len 512, 1M steps, Adam (lr $1e-4$)

Computational cost:

BERT-Base: TPU v3 $\times$ 4 pods $\times$ 4 days

BERT-Large: TPU v3 $\times$ 16 pods $\times$ 4 days

Estimated: **Thousands to tens of thousands of dollars**

Characteristics of Pre-Training

Key point: Pre-training only needs to be done once. By using Google's published pre-trained model, only fine-tuning is needed (1 GPU $\times$ a few hours)

Pre-Training vs Fine-Tuning Cost

	Pre-training	Fine-tuning
Data size	3.3B words	Tens to hundreds of K examples
Compute	TPU $\times$ 4-16 pods	GPU $\times$ 1
Time	Several days	Several hours
Labels	Not required	Required
Runs	Once	Per task

BERT (Encoder)

- **Bidirectional**: Uses both left and right context
- Pre-training: MLM + NSP
- Strengths: Classification, ranking, NER
- Primary use in document ranking: **Reranking** (Cross-encoder)

GPT (Decoder)

- **Unidirectional** (left-to-right): Uses only left context
- Pre-training: Next token prediction
- Strengths: Text generation, summarization, dialogue
- Primary use in document ranking: **Listwise reranking** (RankGPT, etc.)

$$\text{GPT} : \mathcal{L}(\mathcal{U}) = \sum_i \log P(u_i \mid u_{i-k}, \dots, u_{i-1}; \Theta)$$

GPT: next token prediction. BERT: mask prediction — different objectives, same "**pre-train then fine-tune**" paradigm

Bidirectionality advantages: "The [MASK] in Kyoto is beautiful" → BERT uses "beautiful" as context; GPT cannot

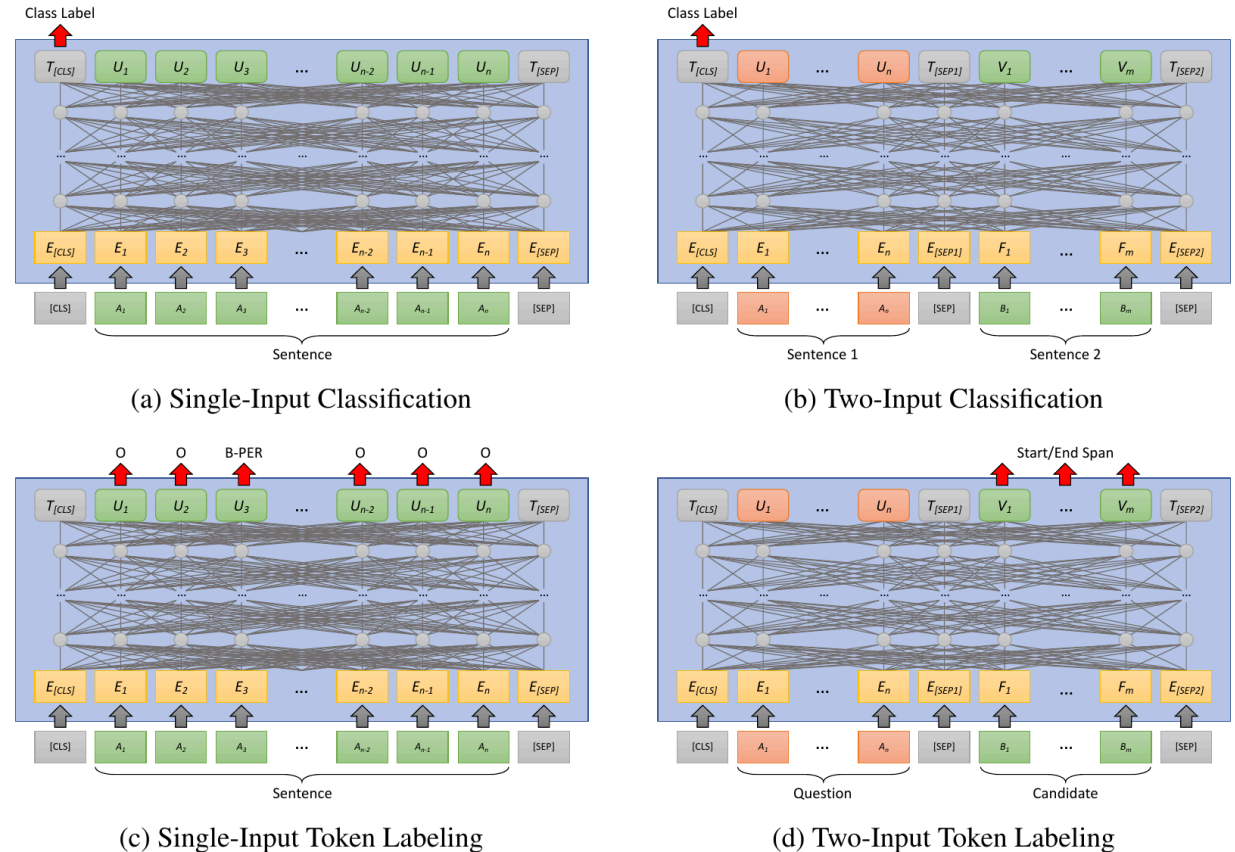
BERT handles **four task types** with the same architecture:

(a) Sentence pair classification: $[CLS] \rightarrow NLI$, document ranking

(b) Single sentence classification: $[CLS] \rightarrow$ Sentiment, spam detection

(c) Question answering: Token outputs $\rightarrow$ Start/end positions (SQuAD)

(d) Token labeling: Per-token labels $\rightarrow$ NER



Source: Lin et al. (2021), Figure 5

Document ranking falls under **(a) sentence pair classification**: Input $[CLS]$ q $[SEP]$ d $[SEP]$ and predict relevance from the $[CLS]$ output. This will be covered in detail with monoBERT in the next lecture

Impact on NLP in general:

Achieved **simultaneous** state-of-the-art on 11 NLP benchmarks (as of 2019)

GLUE benchmark: Results exceeding human performance

Established the paradigm of "one model for all tasks"

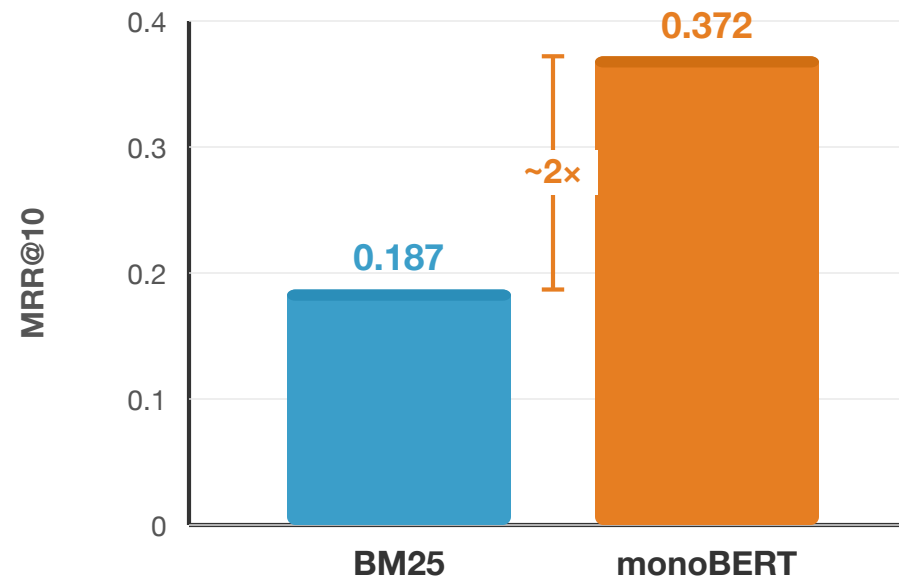
Impact on Information Retrieval (IR):

MS MARCO passage ranking: BM25 (MRR@10 = 0.187) → monoBERT (MRR@10 = 0.372), a **2x performance improvement**

A single model surpassed over 20 years of accumulated IR research

Standardization of the Retrieve-and-Rerank pipeline

MS MARCO Passage Ranking (MRR@10)



Lin et al. (2021), Table 5

MRR@10 values from Lin et al. (2021), Table 5

Architectural constraints:

Max 512 tokens: Cannot process long docs → Birch, BERT-MaxP (Lecture 7)

Computational cost: ~30ms/pair → impractical for full corpus

Cross-encoder: Query-document concatenated → no pre-computation

Learning challenges:

Labeled data needed for fine-tuning; domain shift degrades performance

NSP task found unnecessary in RoBERTa

BERT Constraints and Corresponding Approaches

Constraint	Countermeasure	Lecture
512-token limit	Birch, BERT-MaxP	7
Inference speed	Knowledge distillation, multi-stage	8
Cross-encoder	Bi-encoder (DPR)	10
Lack of labeled data	doc2query, DeepCT	9
No sparse representation	SPLADE	11

Lectures 7–11 cover approaches to overcome these constraints. Understanding BERT's limitations is key to subsequent methods

Training for Ranking

Training for Ranking

Standard recipe:

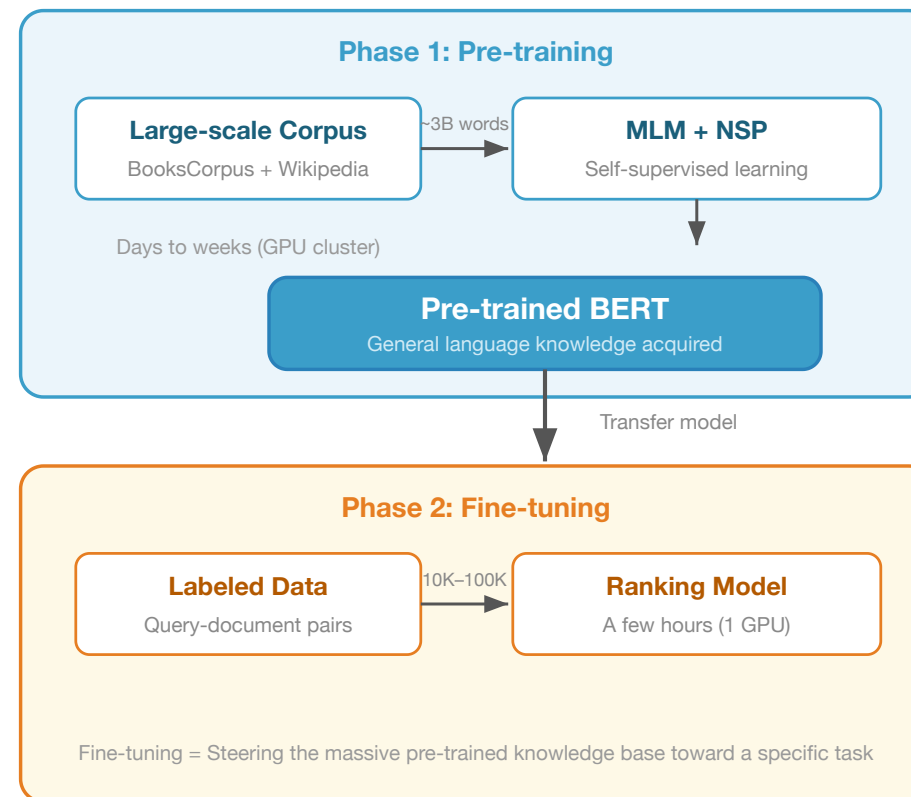
1. **Pre-training:** Learn MLM/NSP on a large corpus

2. **Fine-tuning:** Adjust on labeled data

Leverages linguistic knowledge from pre-training to build high-accuracy models with **small amounts of data**

Pre-training: Billions of words, days to weeks (GPU cluster)

FT: Tens to hundreds of K examples, several hours (1 GPU)



Parameters being updated:

1. **All of BERT** (110M–340M): Fine-tuning the pre-trained weights
2. **Classification head** (new): Task-specific output layer

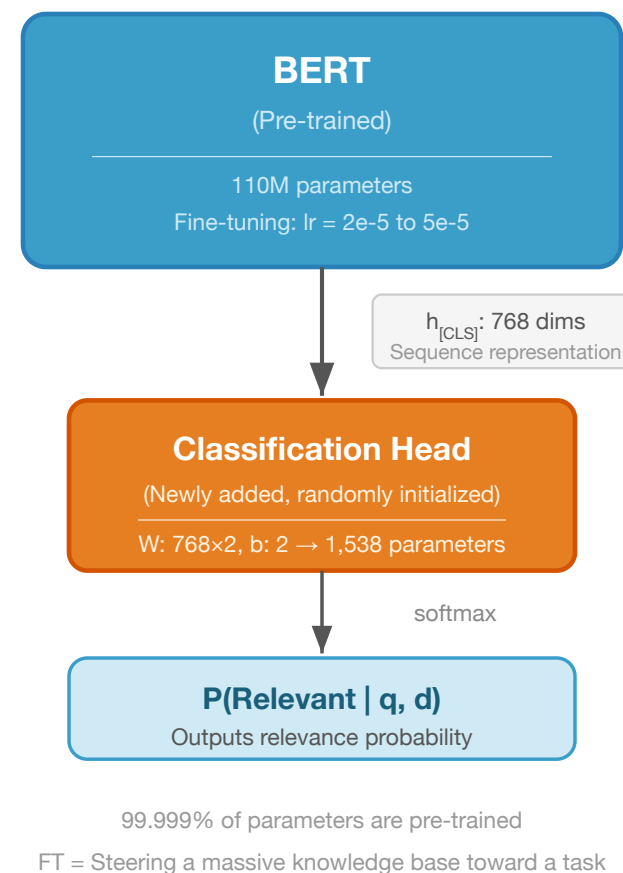
Ranking task output:

$$P(\text{Rel} \mid q, d) = \text{softmax}(h_{[\text{CLS}]}W + b)$$

$h_{[\text{CLS}]}$: [CLS] output (768 dimensions)

$W \in \mathbb{R}^{768 \times 2}$, $b \in \mathbb{R}^2$: Additional parameters (1,538 total)

Learning rate: 2e-5 to 5e-5 (set low to protect existing weights)



99.999% of all parameters are pre-trained. Fine-tuning is the process of "directing" this massive knowledge base

Pointwise: Cross-Entropy Loss

The simplest approach. Binary classification of relevant/non-relevant for each (q, d) pair:

$$\mathcal{L}_{\text{CE}} = -[y \log s + (1 - y) \log(1 - s)]$$

$y \in \{0, 1\}$: Relevance label

$s = P(\text{Relevant} \mid q, d)$: Model output

Pairwise: Margin Loss

Training with pairs of a relevant document d^+ and a non-relevant document d^- :

$$\mathcal{L}_{\text{pair}} = \max(0, \epsilon - s(q, d^+) + s(q, d^-))$$

Comparison of Loss Functions

Loss function	Input	Characteristics
Cross-Entropy	Individual (q, d)	Simple, easy to parallelize
Pairwise Margin	(q, d^+, d^-)	Learns relative order
RankNet	(q, d^+, d^-)	Probabilistic formulation
Listwise	$(q, d_1, \dots, d_k)$	Optimizes the entire list

monoBERT uses **Pointwise CE**. Simple yet highly effective. Pairwise/Listwise often provide limited improvement relative to additional computational cost

The quality of ranking training heavily depends on **what non-relevant documents are selected**

Random negatives:

- Randomly selected from the corpus

- Most examples are too easy → Learning is inefficient

BM25 negatives:

- Non-relevant documents from the top BM25 results

- Examples that are "lexically similar but non-relevant" → More informative

Hard negatives:

- Top non-relevant documents retrieved by other models (DPR, ANCE, etc.)

- The most difficult examples to discriminate → Most effective but also risk of noise

Negative Difficulty and Learning Effect

Easy Negative (Random)

q: "Kyoto autumn leaves" / d: "Quantum mechanics equations"
→ Trivially non-relevant. Little learning signal

Hard Negative (BM25 top)

q: "Kyoto autumn leaves" / d: "Cherry blossom spots in Kyoto"
→ Lexically similar but non-relevant. Strengthens discrimination

The publicly available BERT is pre-trained on a general corpus (Wikipedia + BooksCorpus)

Problem: The target retrieval corpus may differ in vocabulary distribution and genre

Target Corpus Pretraining (TCP):

Perform additional MLM pre-training on the target retrieval corpus **before** fine-tuning

No labeled data required (self-supervised)

"Exposes" the model to the corpus vocabulary and style for domain adaptation

Effect of TCP (MS MARCO Passage)

Method	MRR@10
BM25	0.187
monoBERT	0.372
monoBERT + TCP	0.379

The improvement is modest, but it is a "free" improvement requiring no labels. Larger effects are expected for domain-specific settings (e.g., biomedical)

Domain-specific models such as SciBERT and BioBERT are extensions of the same idea

Strategy for when labeled data for the target task is scarce:

Pre-Fine-Tuning (pFT):

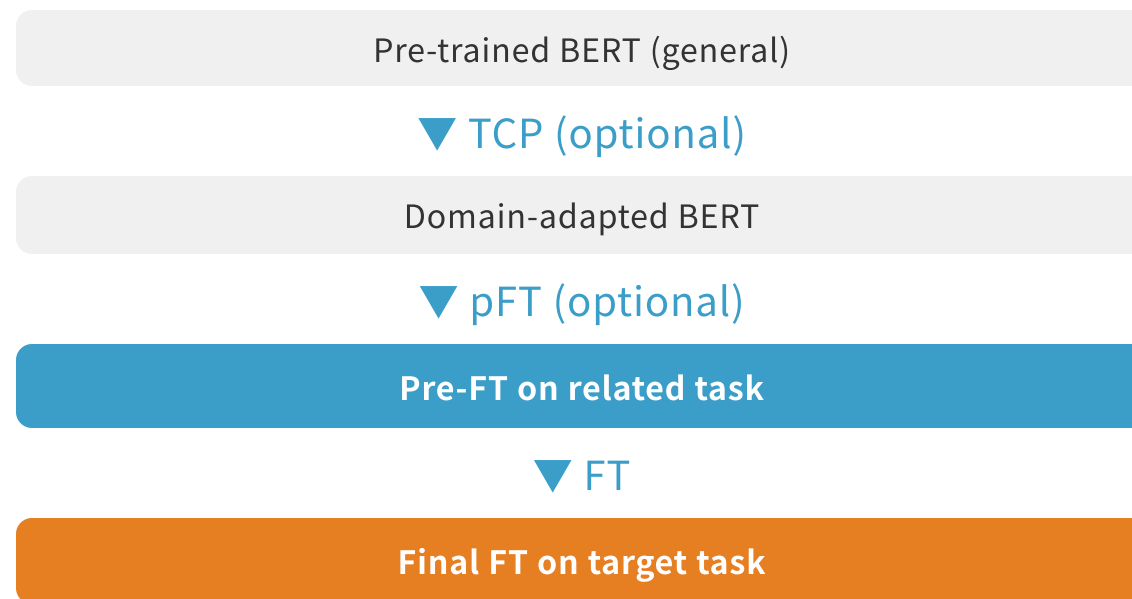
1. Start from a pre-trained BERT
2. **Pre-fine-tune** on data from a related task
3. Final fine-tuning on the target task data

Concrete examples:

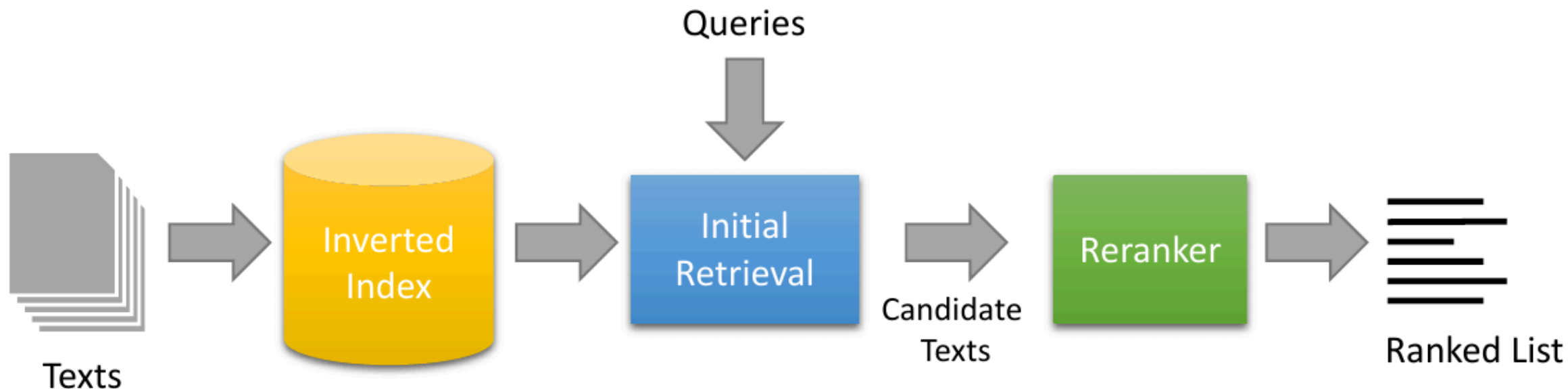
MS MARCO → TREC Robust04: First learn general ranking knowledge, then adapt to a specific collection

NLI data → Ranking: Transfer knowledge of textual entailment between sentences

Overall Training Pipeline



Catastrophic Forgetting: In multi-step FT, training on later datasets risks overwriting knowledge from earlier stages. BERT-based models are relatively robust, but caution is needed



Source: Lin et al. (2021), Figure 6

MS MARCO passage ranking — Typical settings for monoBERT training:

1. **Pre-trained model:** `bert-base-uncased` (Hugging Face)
2. **Training data:** MS MARCO training set (~500K queries x 1 relevant passage each)
3. **Negatives:** Sample non-relevant passages from BM25 top-1,000
4. **Loss function:** Pointwise Cross-Entropy
5. **Optimization:** Adam, learning rate 3e-6, warmup 10%
6. **Batch size:** 32 / **Epochs:** 1-3 (watch for overfitting)

1. **Neural network fundamentals:** Multi-layer perceptrons, activation functions, and embeddings represent words as dense vectors. Evolution from word2vec's static representations to contextualized embeddings
2. **Transformer:** Self-Attention captures context across the entire sequence simultaneously. The combination of Multi-Head Attention, Positional Encoding, residual connections, LayerNorm, and FFN was revolutionary
3. **BERT:** Transformer Encoder + self-supervised pre-training via MLM/NSP. Contextualized embeddings enable polysemy resolution and semantic matching
4. **Fine-tuning:** Specializes a pre-trained model for ranking tasks. Pointwise Cross-Entropy loss, BM25 negative sampling, and TCP are the fundamental techniques

Next Lecture Preview: Neural Ranking Models — Long Document Ranking

Design and ablation experiments of monoBERT

Why BERT is effective for retrieval

Handling long documents: Birch, BERT-MaxP